Stats 506 F25 Midterm

Name: _____

UMID: _____

Question	Out of	Score
1	_____	_____
2	_____	_____
3	_____	_____
4	_____	_____
5	_____	_____
6	_____	_____
Total	_____	_____

Instructions

Complete every problem. Use pen or pencil; closed notes.

Show intermediate work when possible. State any assumptions you make.

Minor syntax mistakes are acceptable when the intended calculation is clear.

Suggested practice time: 75 minutes, matching the F26 exam length.

Practice reconstruction from the official posted solutions. All answers and computed outputs have been removed. Original point allocations were not available in that source, so the score table is left blank.

https://dept.stat.lsa.umich.edu/~jerrick/courses/stat506_f25/midtermsolns.html

Problem 1

Determine q after each independent code fragment runs. Show intermediate work when useful. Assume required packages are loaded.

a.

```
a <- rep(-9, times = 10)
for (i in 1:10) {
  if (i %% 2 == 1) {
    a[i] <- i
  }
}
q <- sum(a[a > 0])
```

q = _____

b.

```
x <- 1:4
y <- matrix(1:12, nrow = 3)
q <- sum(y[, x %% 3 == 0])
```

q = _____

c.

```
x <- c(2, 4, 6, 8)
y <- matrix(c(1, 3, 5, 7, 1, 3, 5, 7), ncol = 2, byrow = FALSE)
q <- x * y
```

q = _____

Problem 1 (continued)

d.

```
q <- sapply(1:10, function(x) {
  ifelse(x < 5, x, x - 5)
})
```

q = _____

e.

```
tibble(x = c(3, 5, 7, 9, 11),
       y = c(2, 2, 3, 3, 3)) %>%
  mutate(z = x + y, w = z %% y, u = z - w) %>%
  group_by(y) %>%
  summarize(v = mean(u) - min(u)) %>%
  arrange(v) %>%
  head(1) %>%
  select(y) %>%
  as.numeric() -> q
```

q = _____

f.

```
foo <- function(x) {
  bar(x + 2)
}
bar <- function(z) {
  baz(z + z^2)
}
baz <- function(t) {
  return(t / 2)
}
q <- bar(5)
```

q = _____

Problem 2

mydata is a data.frame of 4,820 patients with the ten columns below. Write R code for the requested cleaning and analysis steps; create new columns inside mydata and account for missing values.

Variable	Type	Example values
patient_id	int	1 2 3 4 5 6 7 8 9 10 ...
age	int	55 47 61 33 29 74 38 42 68 51 ...
sex	chr	"Male" "Female" "" "Female" "Male" "Male" ...
bmi_category	chr	"Overweight" "Normal" "Obese" "" "Normal" "Obese" ...
years_smoking	int	10 0 25 4 0 40 6 12 30 -2 ...
treatment	chr	"Placebo" "DrugA" "DrugB" "Placebo" "DrugA" ...
systolic_bp	int	132 120 148 115 118 160 128 124 154 139 ...
diastolic_bp	int	85 78 92 70 72 95 80 76 88 82 ...
cholesterol	num	210 180 265 175 190 300 195 205 290 250 ...
outcome_score	num	78 85 NA 90 14 88 82 95 80 91 ...

a. None of the numeric entries should be negative. Recode negative numeric values to NA.

b. Create mean_arterial_pressure, defined as (2 × diastolic_bp + systolic_bp) / 3.

c. Create smoking, indicating nonzero years_smoking.

Problem 2 (continued)

d. Convert sex into a binary variable named female, handling unanswered responses.

e. Restrict the data to patients identified as female.

f. Remove patients outside the inclusive age range 18-65.

g. Rescale outcome_score to the interval from 0 to 1.

h. Fit a linear model of outcome_score using categorical treatment; age and mean_arterial_pressure with their interaction and both main effects; and a cubic relationship in cholesterol.

i. Calculate the average predicted outcome from the model in h.

Problem 3

For a-d, the code is syntactically valid but does not achieve its stated goal. Identify the logical error and fix it with minimal changes.

a. Add a separate random-noise value to every element of x, which may have any length.

```
noisyx <- x + rnorm(1)
```

b. Create yearsempl from yearsfull for full-time workers and yearspart for part-time workers.

```
data$yearsempl <- data$yearsfull
data$yearsempl[data$isparttime] <- data$yearspart
```

c. Add every element of the numeric vector data.

```
for (i in 1:length(data)) {
  sum <- 0
  sum <- sum + data[i]
}
```

d. Estimate average mileage separately for each manufacturer.

```
cars %>%
  select(manufacturer, mpg) %>%
  summarize(meanmpg = mean(mpg))
```

Problem 3 (continued)

For e-g, classify z as always TRUE, always FALSE, or depends, and explain in one or two sentences. mydata is a data.frame and every referenced column exists. Ignore NA outcomes when making the classification.

e.

```
mylist <- as.list(mydata)
z <- length(mylist[[1]]) == length(mylist[[2]])
```

f.

```
mydata$agecapped <- mydata$age
mydata$agecapped[mydata$age > 50] <- 50
z <- mean(mydata$agecapped) > mean(mydata$age)
```

g.

```
mydata2 <- mydata %>%
  filter(urbanrural == "rural") %>%
  group_by(state) %>%
  summarize(meaninc = mean(income))
z <- nrow(mydata) == nrow(mydata2)
```

Problem 4

orders and sku_map contain the rows below. For a-e, give the complete output table, including column names.

orders

p	q	r	k
6	2	1	1
3	5	1	2
9	5	2	4
2	2	3	6

sku_map

k	m
1	10
2	20
3	30
4	40
5	50
6	60

a.

```
SELECT *
FROM orders
WHERE q >= 5
```

b.

```
SELECT k, p + 2 AS p2
FROM orders
ORDER BY -p2
LIMIT 2
```

Problem 4 (continued)

c.

```
SELECT r, SUM(p) AS pp
FROM orders
GROUP BY r
HAVING pp > 7
```

d.

```
SELECT o.p, s.m
FROM orders AS o
INNER JOIN sku_map AS s ON o.k = s.k
```

e.

```
SELECT o.p, s.m
FROM orders AS o
RIGHT JOIN sku_map AS s ON o.k = s.k
```

Problem 4 (continued)

f. Write one SQL query producing this output:

r	k	sku_value
3	6	60
2	4	40
1	2	20
1	1	10

Problem 5

Mark X when each pattern matches the email string; otherwise leave the cell blank. Assume no leading or trailing whitespace. Patterns are displayed as R string contents.

```
a. "^[^@]+@.+\\..+"
b. "(org|com)"
c. "[0-9]{3}"
d. "\\.[a-z]+\\.[a-z]+$"
```

String	a	b	c	d
alice@gmail.com				
bob.smith@umich.edu				
test123@yahoo.com				
jane-doe@company.org				
contact@123web.net				
admin@mailserver.local				
user99@gmail.com				
info@college.edu				
myorg@domain.co.uk				
support@website.io				

Scratch work:

Problem 6

Each mystery pack contains one sticker, labeled A-E. Pack 1 costs $5 and assigns probability 0.20 to each type. Pack 2 costs $2 and assigns probabilities 0.30, 0.30, 0.30, 0.05, 0.05 to A, B, C, D, E respectively.

You may use sample(x, size, replace, prob). It returns size draws from x; replace controls replacement and prob supplies optional relative weights. Input checks and function documentation are not required.

Functions may be inefficient; minor extra credit may reward efficiency.

a. Write open_pack(pack_type), where pack_type is 1 or 2. Return a character vector of length one containing the sticker received.

Problem 6 (continued)

b. Write open_packs(pack_type, n). Return a character vector containing the results of opening n packs of the chosen type. You may call open_pack, but are not required to do so.

Problem 6 (continued)

c. You have $40 and must buy only one pack type: eight Pack 1 packs or twenty Pack 2 packs. Write spend40(pack_type, reps) to estimate the percentage of runs that collect all five types. Use open_pack or open_packs. Compare the two choices and state which is better. The function unique may help.

This uses the $40 budget explicitly corrected in the official solution note. Random estimates need not equal any particular numerical value.